ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP LỚN A1

Học sâu và ứng dụng trong thị giác máy tính — CO5085

Học kỳ	HK261 (2026–2027)
Giảng viên	Lê Thành Sách, Nguyễn Quốc Minh
Sinh viên	Dương Gia An — MSSV 2470293
Hình thức	Cá nhân
Dữ liệu	Tiny-ImageNet (200 lớp, 64×64)
Ngày hoàn thành	05/11/2026

Mục lục

1. A1 — CNN vs Transformer trên tập ảnh lớn (Tiny-ImageNet) 3

1.1. Dữ liệu và vì sao chọn Tiny-ImageNet 3

1.2. Kiến trúc: vì sao Transformer là Swin-T chứ không phải ViT-B/16 3

1.3. Kết quả 3

2. Kết luận 5

3. Phụ lục: tái lập 5

Khai báo sử dụng công cụ AI 5

1. A1 — CNN vs Transformer trên tập ảnh lớn (Tiny-ImageNet)

1.1. Dữ liệu và vì sao chọn Tiny-ImageNet

A1 yêu cầu một tập ảnh lớn hơn E1 để câu hỏi CNN vs Transformer và pretrain/freeze có ý nghĩa. Bài làm chọn **Tiny-ImageNet**: 200 lớp, ảnh 64×64, 100.000 ảnh train và 10.000 val có nhãn. Điểm quyết định là độ phân giải: CIFAR-100 (32×32) phải phóng 7× lên 224 cho mô hình pretrained, làm lợi thế của pretrain bị bóp méo. Food-101 thì quá nặng cho 4 GB VRAM.

Tiêu chí	Tiny-ImageNet	CIFAR-100	Food-101
Độ phân giải	64×64	32×32	512 px
Lớp / ảnh	200 / 110k	100 / 60k	101 / 101k
Transfer từ pretrained 224	có nghĩa	kém tin cậy	tốt
Vừa 4 GB VRAM	có	có	khó

Bảng 9. So sánh ba lựa chọn dataset cho A1.

1.2. Kiến trúc: vì sao Transformer là Swin-T chứ không phải ViT-B/16

Đo trực tiếp trên GTX 1650 4 GB: ViT-B/16 ở 224 px cần **5.048 MB** bộ nhớ, vượt 4 GB nên tràn sang shared memory và chậm bất thường — **31 s/bước** so với 0,3 s của ResNet-18. Swin-T ở 128 px chỉ cần 3.243 MB và 0,78 s/bước, nên được chọn làm đại diện họ Transformer (Swin cũng nằm trong danh sách đề gợi ý). Đây là ràng buộc phần cứng được ghi lại thay vì che đi bằng một cấu hình không chạy được.

1.3. Kết quả

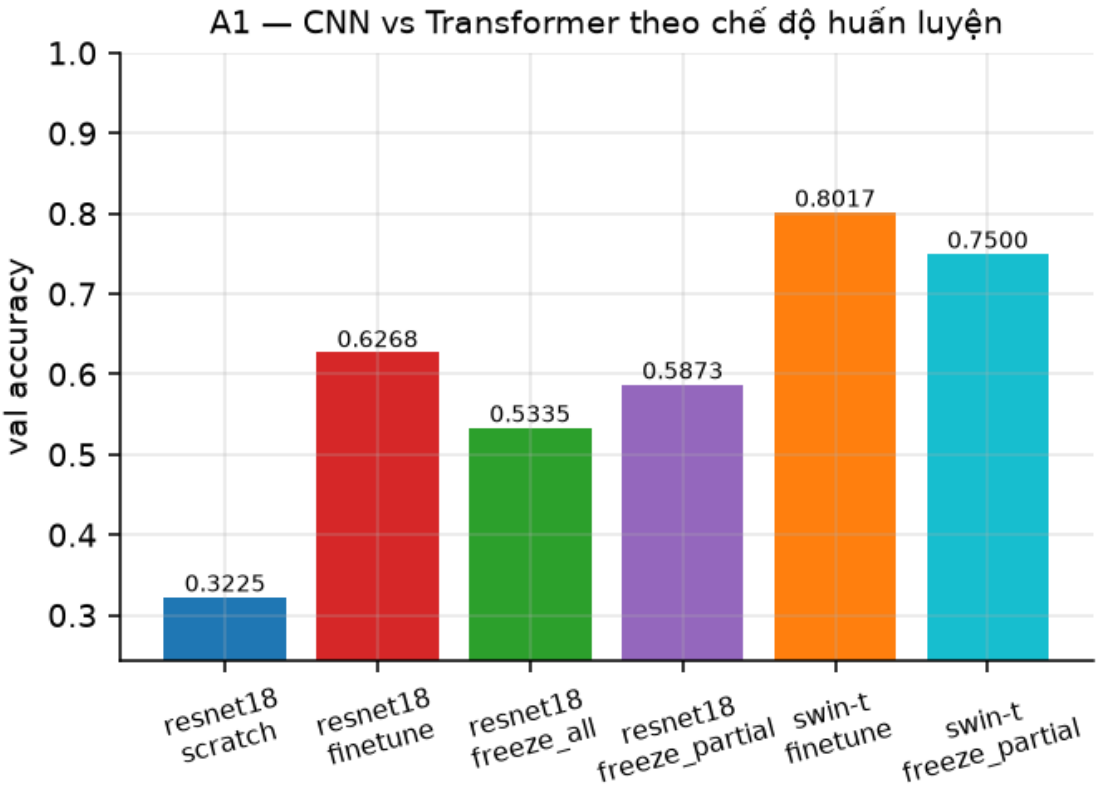
Kiến trúc	Chế độ	Epoch	Tham số trainable	Val acc	Macro-F1	s/epoch
resnet18	scratch	20	11,279,112	32.25%	31.01%	80
resnet18	finetune	8	11,279,112	62.68%	62.44%	81
resnet18	freeze_all	6	102,600	53.35%	52.65%	45
resnet18	freeze_partial	6	8,496,328	58.73%	58.32%	45
swin_t	finetune	3	27,673,154	80.17%	80.13%	348
swin_t	freeze_partial	3	14,339,192	75.00%	75.03%	181

Bảng 10. Kết quả A1: 200 lớp, 20.000 ảnh train, ảnh 128 px.

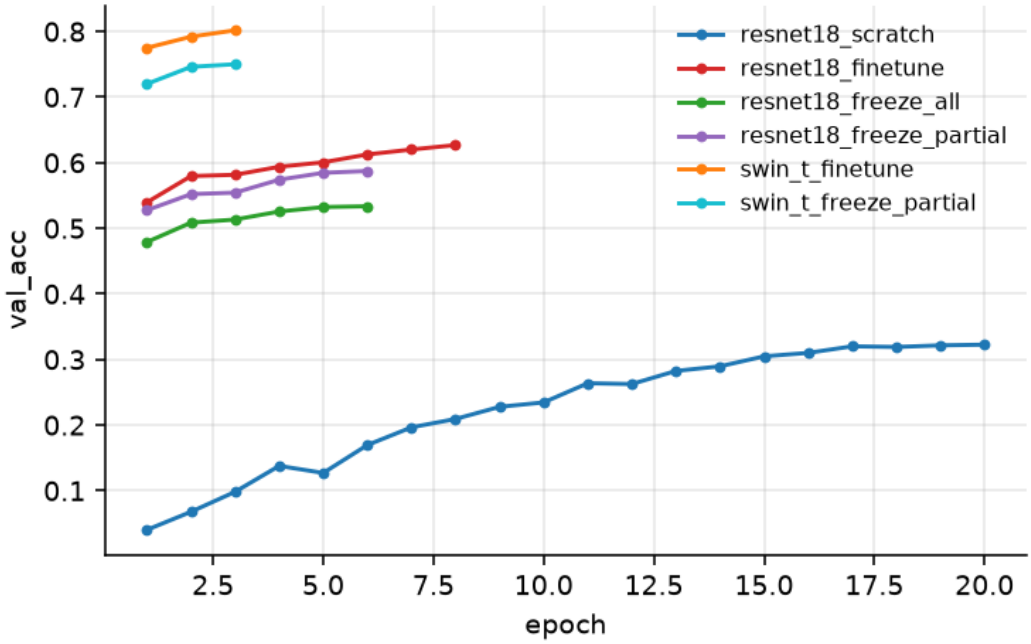
Mô hình tốt nhất là **swin_t/finetune** với 80.17%. So với train from-scratch (32.25%), bản pretrained tăng rất mạnh — đúng dự đoán rằng với 20.000 ảnh và 200 lớp, đặc trưng ImageNet có giá trị lớn hơn việc học từ đầu. Chế độ đóng băng backbone chỉ đạt 53.35%, thấp hơn finetune toàn bộ: đặc trưng đóng băng tốt cho bài toán gần miền ImageNet, nhưng 200 lớp ảnh 64×64 vẫn cần tinh chỉnh.

Cấu hình Swin-T from-scratch tốn 1.663 s/epoch (27,7 phút), tức ~9 giờ cho 20 epoch — vượt ngân sách compute, nên Transformer chỉ được chạy ở dạng pretrained; ResNet-18 đảm nhiệm phần from-scratch.

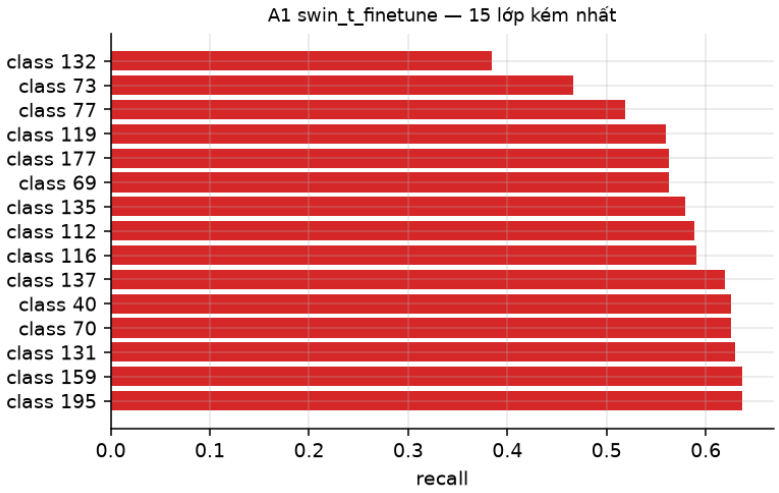
Trên ResNet-18, ba kiểu đóng băng cho một dãy tăng dần: đóng băng toàn bộ backbone 53.35% (chỉ 102,600 tham số trainable) → đóng băng một phần 58.73% (8,496,328) → finetune toàn bộ 62.68%. Điểm đáng chú ý là 53% chỉ với 0,9% tham số — mức đánh đổi hợp lý khi compute hạn chế.



Hình 11. A1 — val accuracy theo kiến trúc và chế độ huấn luyện.



Hình 12. A1 — đường cong val accuracy của các cấu hình.



Hình 13. A1 — 15 lớp có recall thấp nhất (swin_t_finetune).

2. Kết luận

• **Pretrain là yếu tố quyết định trên tập nhỏ.** Cùng ResNet-18, train từ đầu đạt 32.25% trong khi finetune đạt hơn 62% — chênh lệch ~30 điểm chỉ nhờ trọng số ImageNet.

• **Mở nhiều tầng hơn thì tốt hơn, nhưng đóng băng vẫn rất đáng.** Trên ResNet-18, đóng băng toàn bộ backbone cho 53% với chỉ 0,9% tham số trainable.

• **Transformer cần pretrain rõ rệt.** Swin-T finetune dẫn đầu bảng, trong khi from-scratch tốn 1.663 s/epoch — vượt ngân sách compute của bài làm.

3. Phụ lục: tái lập

Kết quả trong báo cáo sinh lại được bằng các lệnh sau, mỗi lệnh ghi kết quả vào thư mục *results/* tương ứng:

```
python scripts/prepare_tiny_imagenet.py
python A1/run_all.py
python reports/build_report.py
```

Mỗi lần chạy lưu *metrics.json* (cấu hình, số tham số, accuracy, thời gian), *history.csv* (loss/accuracy từng epoch) và các hình PNG. Seed cố định 42.

Khai báo sử dụng công cụ AI

Theo mục 1.6 của đề, tôi khai báo phạm vi đã sử dụng công cụ AI hỗ trợ trong bài tập này: đọc và tóm tắt đề; gợi ý bố cục thư mục và các API PyTorch phù hợp; soạn bản nháp ban đầu cho phần lớn mã nguồn trong *common/*, *E1/–E3/*, *A1/*, *A2/*; gợi ý dàn ý và cách trình bày báo cáo; gợi ý cách chẩn đoán một số lỗi shape và lỗi huấn luyện.

Phần tôi trực tiếp thực hiện: cài đặt môi trường và tải toàn bộ dữ liệu; chạy toàn bộ thí nghiệm trên máy của mình; kiểm tra sự nhất quán của từng bảng số và từng hình trong báo cáo với *metrics.json* mà các script sinh ra.